

Quoridor – Python

Rapport final

AIT-ISSAD MOHAMED SAID, BILAL AL FAYOUMI, AMARA KOUSAY, GUILHEM CAUSSE, OMAR HARCHI

9 avril 2026

Introduction

Quoridor est un jeu de société à deux ou quatre joueurs conçu par Mirko Marchesi et publié par Gigamic en 1997. Sur un plateau de neuf cases de côté, chaque joueur doit conduire son pion de son bord de départ au bord opposé, tout en posant des murs pour ralentir ses adversaires. La règle fondamentale qui interdit de condamner tout chemin vers la victoire confère au jeu une profondeur stratégique remarquable : l'espace des états avoisine 10^{42} configurations et la complexité de l'arbre de jeu dépasse 10^{162} , plaçant Quoridor parmi les jeux combinatoires les plus exigeants pour les méthodes de recherche automatique.

Ce projet a pour objectif de réaliser une implémentation complète de Quoridor en Python, allant du moteur de jeu jusqu'aux interfaces utilisateur, en passant par plusieurs algorithmes d'intelligence artificielle. Le moteur repose sur une modélisation par graphe : les murs sont représentés comme des suppressions d'arêtes, et chaque pose de mur est validée par un parcours en largeur vérifiant que tous les chemins restent praticables. Sur cette base, trois algorithmes d'IA ont été développés : un Minimax avec élagage alpha-bêta et approfondissement itératif, un Monte Carlo Tree Search guidé par une politique UCT, et un pipeline d'apprentissage automatique fondé sur une forêt aléatoire intégrée à la sélection du MCTS. Ces algorithmes partagent le même cœur de jeu et sont interchangeables à l'exécution.

Deux interfaces ont été développées au-dessus de cette logique commune. Une interface en ligne de commande (**CLI**) permet de jouer en mode texte, de configurer l'IA et de gérer les parties sauvegardées. Une interface graphique (**GUI**) réalisée avec GTK4 et le moteur de rendu Cairo offre un plateau interactif avec glisser-déposer des pions, placement visuel des murs, gestion du temps en mode Blitz et connexion à un serveur réseau. La séparation stricte entre la couche de présentation et la logique applicative garantit que les deux interfaces s'appuient sur exactement le même moteur de jeu.

Ce rapport présente l'ensemble de la démarche. La section 1 décrit le jeu, ses règles et sa complexité. La section suivante expose la modélisation retenue, puis l'architecture logicielle générale. La conception des algorithmes d'intelligence artificielle est ensuite détaillée, suivie de l'implémentation de l'interface graphique. Le rapport se conclut par une présentation des tests réalisés et une revue critique du projet.

1 Explication détaillée du sujet

1.1 Historique du jeu

Quoridor est un jeu de stratégie abstraite conçu par le créateur de jeux italien Mirko Marchesi et publié en 1997 par l'éditeur français Gigamic [?, ?]. Son histoire remonte à un jeu antérieur, *Blockade*, inventé par Philip Slater en 1975 [?] : ce dernier posait déjà le principe consistant à gêner la progression adverse au moyen d'obstacles disposés sur un plateau en grille. Marchesi s'inspira de cette mécanique pour concevoir en 1995 une première variante, *Pinko Pallino*, éditée par la société italienne Epta Games [?]. Pinko Pallino se jouait à deux joueurs sur un plateau de 11×11 cases avec 42 murs au total et des règles légèrement différentes. Quoridor en constitue la formulation ultérieure, ramenée à un plateau de 9×9 cases et ouverte à deux ou quatre joueurs.

Dès sa sortie, le jeu obtient une reconnaissance critique internationale : il reçoit le *Mensa Select Award* en 1997 [?], puis est élu *Jeu de l'Année 1998* par le magazine américain *Games Magazine* dans la catégorie des jeux de stratégie abstraite [?]. Ces distinctions consacrent sa qualité de conception : des règles très accessibles combinées à une profondeur tactique surprenante. De nombreux établissements d'enseignement européens, notamment dans le cadre du réseau MindLab, ont depuis intégré Quoridor comme support pédagogique pour l'enseignement de la logique et de l'al-

gorithmique [?]. Le jeu est aujourd'hui décliné en de nombreuses versions (Pocket, Kid, Deluxe, Giant) et continûment édité à l'échelle mondiale (voir figure 1).



FIGURE 1 – Boîte du jeu Quoridor (Gigamic, 1997).

1.2 Règles du jeu

Quoridor se joue sur un plateau carré de 9×9 cases. Chaque joueur positionne son pion au centre du bord qui lui est assigné et dispose d'une réserve de murs : dix murs par joueur en partie à deux, cinq en partie à quatre [?]. L'objectif est d'atteindre en premier l'une des neuf cases du bord opposé à son bord de départ.

À chaque tour, un joueur accomplit exactement l'une des deux actions suivantes.

Déplacement du pion : le pion se déplace d'une case dans l'une des quatre directions orthogonales (nord, sud, est, ouest), sous réserve qu'aucun mur ne barre le passage et que la case d'arrivée soit libre. Lorsque le pion adverse se trouve sur une case immédiatement adjacente sans mur interposé, le joueur peut sauter par-dessus pour atterrir sur la case située juste derrière l'adversaire. Si cette case est inaccessible (mur ou bord du plateau), le saut peut s'effectuer en diagonale sur l'une des cases latérales adjacentes à l'adversaire [?, ?] (voir figure 2).

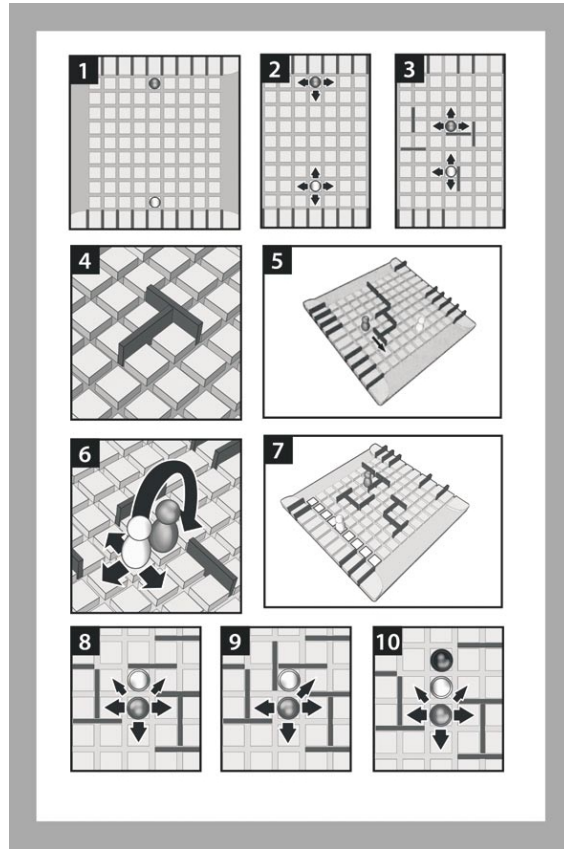


FIGURE 2 – Illustrations des règles de déplacement, tirées du manuel officiel [?].

Pose d'un mur : un mur est une pièce plate couvrant deux segments consécutifs de la grille, posé dans l'une des rainures du plateau. Il bloque définitivement le passage entre les cases qu'il sépare, pour tous les joueurs. La règle la plus importante du jeu — et la plus exigeante du point de vue algorithmique — interdit de poser un mur si ce placement supprime *le dernier chemin* permettant à l'un quelconque des joueurs d'atteindre son bord objectif [?, ?]. Cette contrainte impose, à chaque tentative de pose, une vérification de connectivité par un parcours de graphe (voir section 2).

Exemples illustratifs : pour ancrer ces règles dans la pratique, considérons quelques situations concrètes exprimées en notation algébrique [?].

Déplacement simple : le joueur 1 part de la case **e9** (centre du bord sud) et avance vers le nord : il atteint **e8**. Si un mur horizontal a été posé en **e8h**, bloquant le passage entre les rangées 8 et 9, ce déplacement est illégal et le joueur doit choisir une autre direction ou poser un mur à la place.

Saut et repli diagonal : le joueur 1 est en **e5** et le joueur 2 en **e4**, sans mur entre eux. Le joueur 1 peut sauter directement en **e3**. Si un mur horizontal bloque **e3**, le saut droit est impossible : le joueur 1 peut alors se rabattre en diagonale vers **d4** ou **f4**, à condition qu'aucun mur ne barre ces accès latéraux [?] (voir figure 3).

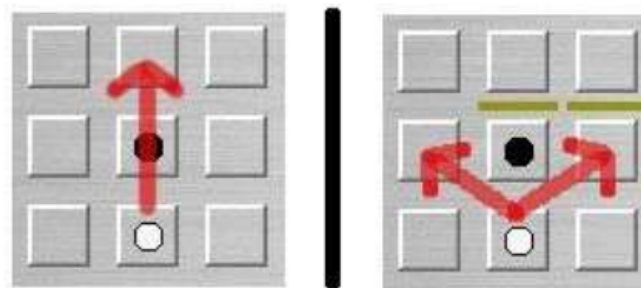


FIGURE 3 – Saut direct par-dessus l'adversaire (gauche) et repli diagonal lorsqu'un mur bloque l'atterrissage (droite) [?].

Mur illégal : supposons que des murs successifs aient été posés de façon à ne laisser qu’un seul chemin au joueur 2 vers son bord objectif. Toute tentative de poser un mur supplémentaire supprimant ce dernier chemin est rejetée par le moteur de jeu, qui déclenche un parcours en largeur (BFS) sur le graphe du plateau pour détecter la violation avant d’enregistrer l’action (voir section 2). Cet exemple illustre directement pourquoi la vérification de connectivité est au cœur de l’implémentation.

1.3 Classification et complexité du jeu

Quoridor appartient à la famille des jeux combinatoires abstraits à information parfaite, déterministes et à somme nulle, au même titre que les Échecs ou le Go [?]. Cette classification implique que les techniques de recherche adversariale classiques sont théoriquement applicables : en particulier, l’élagage alpha-bêta introduit par Knuth et Moore [?] permet d’explorer l’arbre de jeu en réduisant significativement le nombre de nœuds évalués.

Cependant, la taille de l’espace d’états est considérable. Mertens estime le nombre de positions légalement atteignables à environ $3,9 \times 10^{42}$, et la complexité de l’arbre de jeu autour de 10^{162} [?], soit un ordre de grandeur supérieur à celui des Échecs (environ 10^{123} [?]). Glendenning montre que le facteur de branchement moyen est de l’ordre de 60 à 80 coups légaux par position [?], dominé par les nombreux placements de murs possibles (jusqu’à 128 en début de partie). Une recherche exhaustive est donc totalement hors de portée, et la qualité d’un agent repose essentiellement sur la pertinence de sa fonction d’évaluation heuristique.

Ces caractéristiques sont résumées dans le tableau 1 et ont un impact direct sur nos choix d’implémentation, détaillés en section 2 : nécessité d’un élagage alpha-bêta avec approfondissement itératif, coût prohibitif d’un BFS répété pour la vérification de légalité des murs, et importance d’une représentation interne du plateau permettant l’ajout et la suppression rapides d’arêtes.

Critère	Quoridor	Échecs	Go (19 × 19)
Espace d’états	$\approx 10^{42}$	$\approx 10^{44}$	$\approx 10^{170}$
Complexité arbre de jeu	$\approx 10^{162}$	$\approx 10^{123}$	$\approx 10^{360}$
Facteur de branchement	60–80	35	250
Information	parfaite	parfaite	parfaite
Hasard	non	non	non
Match nul	impossible	possible	possible

TABLE 1 – Comparaison de la complexité de Quoridor avec d’autres jeux combinatoires de référence [?, ?, ?].

1.4 Travaux existants

Quoridor a fait l’objet de plusieurs travaux académiques qui constituent le socle bibliographique de notre projet.

Approches par recherche adversariale : le premier agent documenté est dû à Lisa Glendenning, dont la thèse de licence (University of New Mexico, 2005) propose un joueur artificiel fondé sur un algorithme Minimax avec une fonction d’évaluation linéaire dont les poids sont optimisés par algorithme génétique [?]. La fonction d’évaluation privilégie la différence de longueurs des plus courts chemins des deux joueurs vers leurs lignes d’arrivée respectives, quantité qui s’avère être le signal heuristique le plus discriminant. Glendenning propose également la première notation algébrique formelle pour Quoridor, analogue à la notation des Échecs.

Mertens complète cette approche en 2006 en introduisant des heuristiques supplémentaires basées sur la progression colonne par colonne et en analysant l’impact du choix des poids sur les performances [?]. Son agent repose sur Minimax à profondeur 2 et calcule la connectivité par une recherche en profondeur d’abord (DFS) et les plus courts chemins par BFS. Il conclut que la richesse heuristique prime sur la profondeur de recherche pour des temps de réflexion raisonnables.

Approches Monte Carlo : plus récemment, Massagué Respall, Brown et Aslam ont appliqué la recherche arborescente Monte Carlo (MCTS) à Quoridor [?]. Leur agent repose sur l’algorithme UCT (*Upper Confidence bounds applied to Trees*) formalisé par Kocsis et Szepesvári [?], qui équilibre exploration et exploitation au sein de l’arbre de jeu. Les résultats montrent que MCTS se révèle compétitif face à un Minimax bien réglé, et que l’incorporation d’heuristiques dans les phases de sélection et de simulation améliore sensiblement les performances. Cette étude confirme que la modélisation par graphe et les algorithmes de plus court chemin constituent le socle commun de toutes les implémentations sérieuses du jeu. Browne *et al.* fournissent le cadre de référence sur les variantes MCTS [?].

Positionnement de notre projet : les solutions logicielles existantes — agents web universitaires, applications mobiles multijoueurs — ne réunissent pas simultanément les caractéristiques suivantes : logiciel libre, interfaces CLI et GUI, mode réseau multijoueur et intelligence artificielle configurable. C’est précisément le créneau que vise notre projet, en nous appuyant sur les algorithmes validés par les travaux cités ci-dessus (voir section 2).

2 Modélisation du jeu

2.1 Représentation de l'état du jeu

L'état d'une partie est centralisé dans `GameState`. La classe regroupe les positions des joueurs, les murs restants et posés, le graphe de connectivité et le joueur courant. Cette séparation entre état dynamique, contraintes de déplacement et progression du tour évite de disperser la logique métier.

Les positions utilisent un encodage linéaire :

$$node = row \times size + col.$$

Ce format simplifie l'accès aux structures indexées et les parcours. Les champs utilisés en exécution sont `board_size`, `current_player`, `player_positions`, `remaining_walls`, `vertical_walls`, `horizontal_walls` et `graph`. Cette représentation sert directement la validation des coups, l'historisation et la persistance.

2.2 Graphe du plateau et structures associées

Le plateau est modélisé par un graphe non orienté $G = (V, E)$: les sommets représentent les cases et les arêtes les déplacements élémentaires autorisés, conformément à [?, ?]. Le graphe est stocké en liste d'adjacence `dict[int, list[int]]`, adaptée à une grille creuse (degré maximal 4) avec un coût mémoire en $O(V + E)$.

Les murs sont appliqués comme suppressions d'arêtes plutôt que comme transformations géométriques [?]. La vérification de connectivité repose sur BFS, avec une complexité $O(V + E)$ sur graphe non pondéré [?]. Les distances de plus court chemin sont aussi réutilisées par les heuristiques IA, notamment pour Minimax et MCTS [?, ?].

En complément, le moteur maintient `player_positions`, deux collections de murs (vertical/horizontal) et un historique de snapshots pour *undo/redo*.

2.3 Déplacements des pions et validation des murs

Les déplacements de base sont obtenus à partir des voisins du sommet courant dans le graphe. Lorsqu'un pion adverse occupe un sommet adjacent, la règle de saut s'applique si la case située derrière reste accessible ; sinon, des déplacements diagonaux sont autorisés selon les arêtes effectivement présentes. Ces cas dépendent de l'état courant des pions et sont traités dynamiquement dans `pawn_rules.py`.

La validation d'un mur combine deux contrôles : validité locale de placement (position, orientation, absence de chevauchement) et maintien d'un chemin vers la zone cible pour chaque joueur. L'implémentation effectue une suppression temporaire des arêtes concernées, exécute un BFS par joueur, puis valide la pose ou restaure l'état précédent. Sur une grille 9×9 , ce coût reste faible.

Le modèle retenu établit un lien direct entre structures de données et règles d'exécution : le graphe représente la connectivité, les règles dynamiques gèrent les cas contextuels et les vérifications globales assurent la validité de la partie.

3 Architecture logicielle

3.1 Architecture générale

Les Figures 4 et 5 présentent les composants principaux du projet en deux vues lisibles.

Vue UML du noyau metier et de la couche applicative

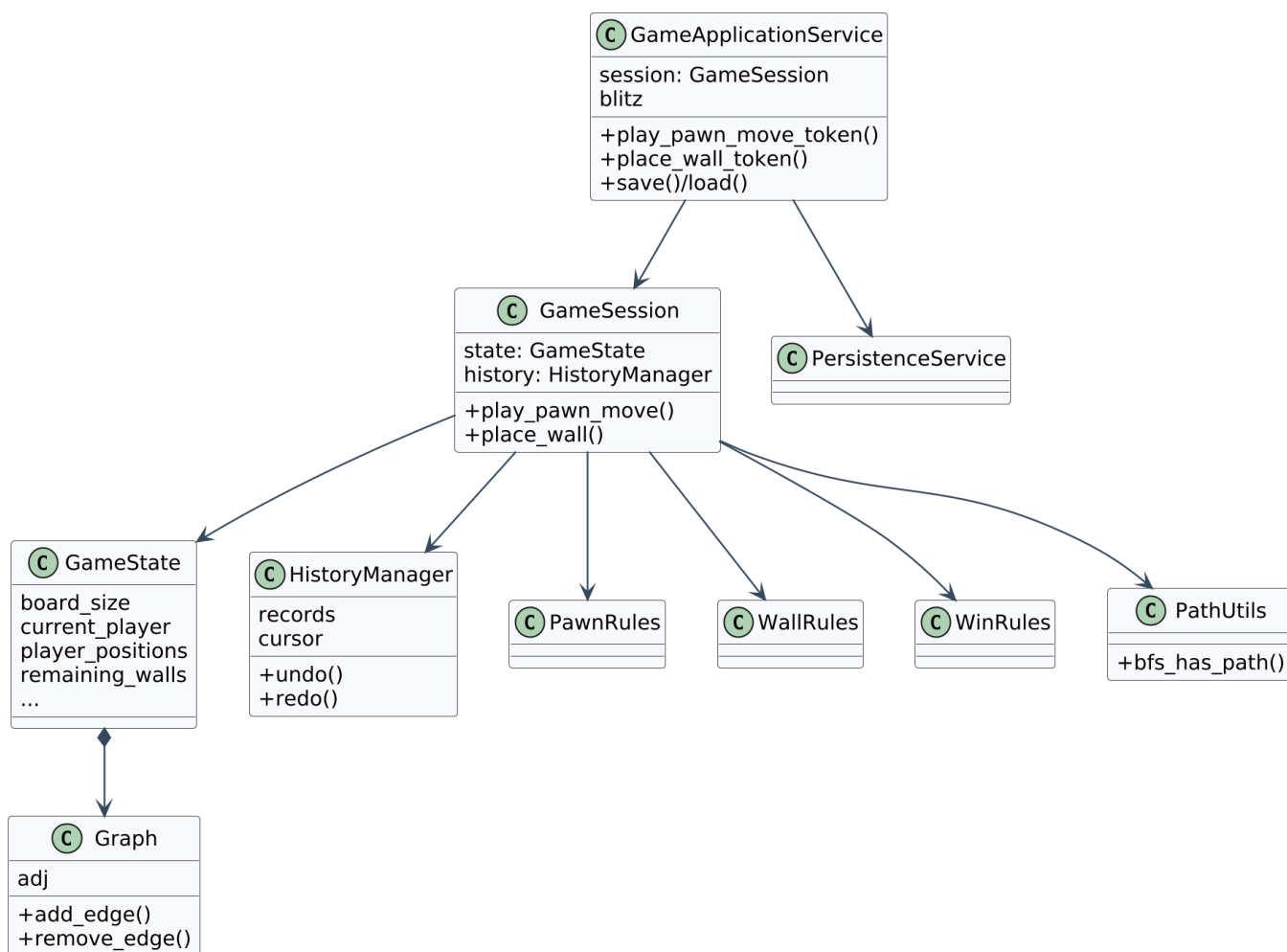


FIGURE 4 – Vue UML du noyau métier et de la couche applicative

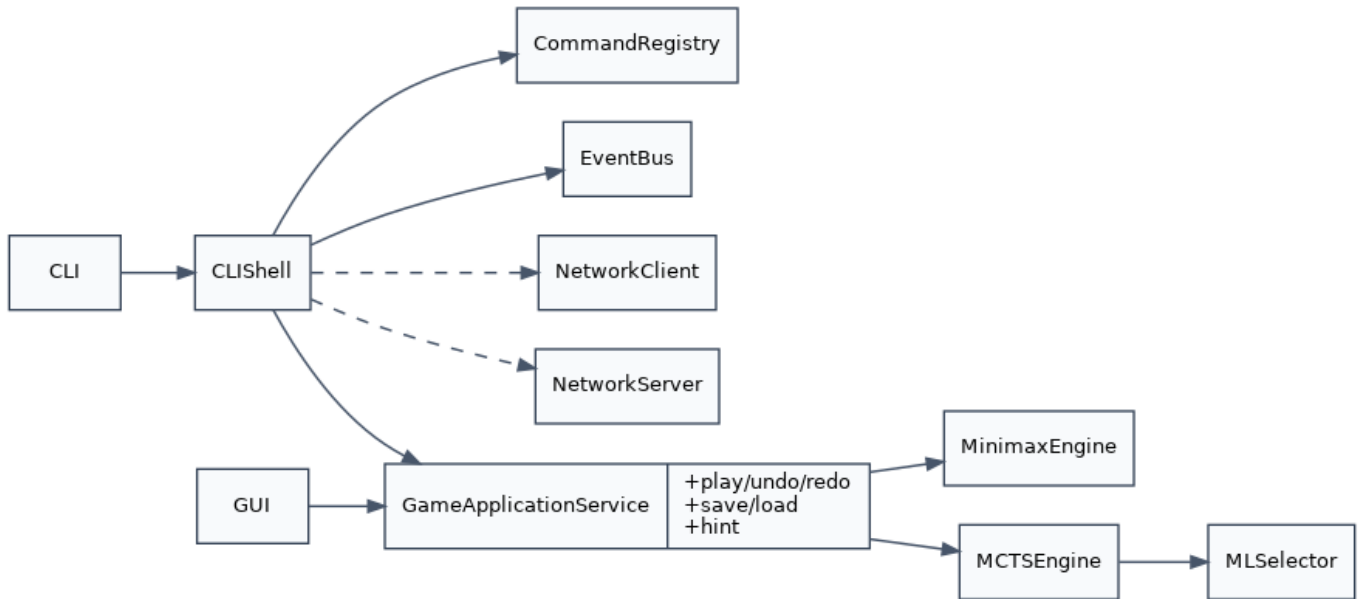


FIGURE 5 – Vue UML des interfaces, du réseau et des composants IA

L'architecture est organisée en couches : les modules d'interface et de réseau passent par la couche applicative, qui orchestre les opérations métier. **GameState** porte l'état, **GameSession** applique les transitions et **GameApplicationService** constitue le point d'entrée commun pour CLI, GUI, IA, persistance et réseau.

3.2 Modules et dépendances

Le module **core** contient le modèle métier (état, plateau, notation, snapshots). Le module **rules** implémente les validations de déplacements, de murs et de victoire. Le module **application** orchestre les cas d'usage et le cycle de partie. Les modules **interfaces** et **network** gèrent l'entrée/sortie utilisateur et les échanges client/serveur. Le module **utils** regroupe les composants techniques réutilisables (graphe, utilitaires), et **ml** porte les composants d'aide aux moteurs de recherche.

Les dépendances restent dirigées : **interfaces** → **application** → (**core**, **rules**, **utils**), avec **network** connecté à **application**. Cette orientation limite les cycles et isole le coeur métier.

Architecture simple de quoridor/

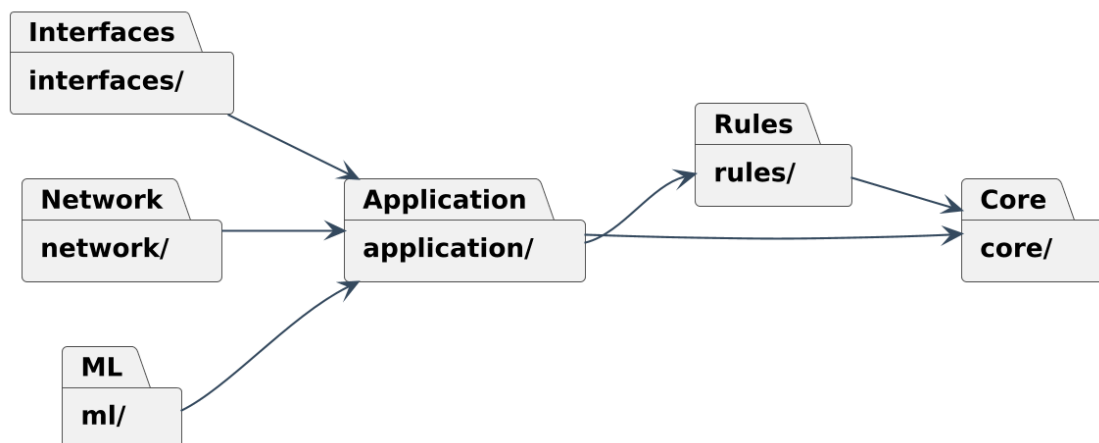


FIGURE 6 – Architecture modulaire de quoridor/ et dépendances principales

3.3 Cheminement des actions

Toutes les actions passent par le même pipeline applicatif. Pour un déplacement utilisateur : interface → **GameApplicationService** → validation **rules** → mise à jour **GameSession/GameState** → historisation via **HistoryManager** → retour interface.

La pose de mur suit la même chaîne, avec simulation sur graphe et test BFS avant validation. Un coup IA suit également ce pipeline : `GameApplicationService` déclenche Minimax/MCTS, puis applique le coup sélectionné via les mêmes étapes de validation et de mise à jour.

3.4 Intégration des algorithmes

BFS est implémenté dans `rules/utils` pour les tests de connectivité. Les moteurs Minimax, alpha-bêta et MCTS sont orchestrés par `application`. Le mécanisme *undo/redo* repose sur `HistoryManager` et les snapshots d'état. La persistance et le parsing assurent la conversion entre formats externes et représentation interne, sans dupliquer la logique métier.

3.5 Design patterns utilisés

Command Dans le shell CLI, les commandes et `CommandRegistry` encapsulent les actions utilisateur et délèguent leur exécution à `GameApplicationService`. Cette structure découple saisie et exécution.

Strategy `PlayCommandStrategy`, `LocalPlayCommandStrategy` et `NetworkPlayCommandStrategy` isolent les variantes d'exécution local/réseau, sans multiplier les conditions dans la couche applicative.

Memento `GameSnapshot`, `MoveRecord` et `HistoryManager` conservent les états avant/après action, ce qui permet des restaurations cohérentes pour *undo/redo*.

Facade `GameApplicationService` joue le rôle de façade : il expose une API unique pour jouer un coup, poser un mur, gérer l'historique, piloter l'IA et charger/sauvegarder une partie.

Observer `EventBus` implémente un mécanisme d'observation léger entre producteurs et consommateurs d'événements, utile pour propager les mises à jour sans dépendances directes fortes.

L'ensemble produit une architecture compacte : responsabilités localisées, dépendances orientées vers le métier, et extensions (IA, réseau, interfaces) ajoutées sans modifier le noyau de règles.

4 Intelligence artificielle

4.1 Minimax avec élagage Alpha-Beta

Le Minimax explore récursivement l'arbre de jeu. À chaque nœud, il génère les coups légaux, applique chaque coup sur une copie de l'état, puis évalue les positions obtenues. Une victoire pour l'IA vaut +1 000 000, une défaite -1 000 000. À profondeur nulle, l'évaluation repose sur l'une des trois heuristiques configurables.

Sans élagage, le coût est exponentiel en fonction du facteur de branchement b et de la profondeur d : $O(b^d)$. Au Quoridor, ce facteur est élevé car il faut considérer à la fois les déplacements de pion et les poses de murs. L'élagage alpha-beta réduit ce coût en supprimant les branches qui ne peuvent plus influencer le résultat. La figure 7 montre ce cas simple : si MAX possède déjà une branche à 5, une branche adverse qui descend à 2 est coupée.

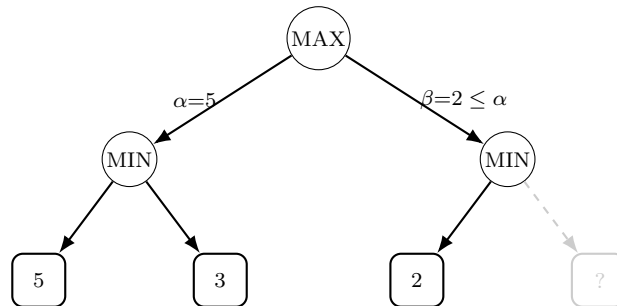


FIGURE 7 – Exemple d'élagage alpha-beta.

Dans les modes à temps borné, la recherche vérifie une deadline avant chaque expansion de nœud. Si le temps est écoulé, elle s'arrête et renvoie le meilleur coup déjà calculé. Le pseudo-code correspondant est donné en figure 8.

```

fonction minimax(etat, profondeur, alpha, beta, maximise):
    verifier la deadline
    si etat terminal: retourner score final
    si profondeur = 0: retourner heuristique(etat)

    pour chaque coup legal:
        evaluer recursivement l'etat enfant
        mettre a jour la meilleure valeur courante
        mettre a jour alpha ou beta
        si beta <= alpha: arreter l'exploration

    retourner la meilleure valeur

```

FIGURE 8 – Pseudo-code du Minimax avec élagage alpha-beta

4.2 Iterative Deepening

Une fois le Minimax avec élagage alpha-beta en place, nous avons été confrontés à un problème pratique : quelle profondeur choisir ? Fixer une profondeur à l'avance semble raisonnable, mais au Quoridor le coût d'exploration varie énormément d'un tour à l'autre. En début de partie, chaque joueur dispose de dix murs : le facteur de branchement est alors très élevé, car l'IA doit évaluer à la fois les déplacements de pion et toutes les poses de murs possibles. En fin de partie, quand les murs sont épuisés, seuls les déplacements restent, et l'arbre est beaucoup plus étroit. Avec une profondeur fixe, on risque donc soit de dépasser le temps alloué en début de partie, soit de gaspiller du temps de calcul en fin de partie.

Pour résoudre ce problème, nous avons implémenté l'**Iterative Deepening**. Le principe est le suivant : plutôt que de lancer une seule recherche à profondeur fixe, on enchaîne plusieurs recherches successives à profondeur croissante — d'abord à profondeur 1, puis 2, puis 3, et ainsi de suite. Avant chaque itération, on vérifie qu'il reste suffisamment de temps. Dès que la deadline est atteinte en cours d'exploration, on interrompt proprement la recherche et on retourne le meilleur coup trouvé lors de la *dernière profondeur complètement terminée*. Ce mécanisme est essentiel : il garantit qu'on ne retourne jamais un coup issu d'une exploration incomplète, tout en exploitant au mieux le temps disponible.

En pratique, nous avons exposé deux modes d'utilisation à l'utilisateur :

- **Profondeur explicite** (`-ai-minimax-depth`) : l'utilisateur fixe lui-même la profondeur de recherche. Le Minimax s'exécute exactement jusqu'à cette profondeur, sans mécanisme de temps. C'est utile pour reproduire des comportements prévisibles ou faire des tests comparatifs.
- **Temps limité** (`-ai-time`) : l'utilisateur fixe un budget temps en secondes. L'Iterative Deepening approfondit la recherche tant que le temps le permet, ce qui donne toujours le meilleur coup calculable dans le délai imparti.

Nous avons également prévu un troisième cas : l'utilisateur choisit le mode `minimax` sans préciser de profondeur ni de temps particulier. Dans cette situation, le programme ne laisse pas l'IA sans repère : il convertit automatiquement le temps par défaut en une profondeur maximale équivalente, selon les seuils définis dans le tableau ci-dessous. Ces seuils ont été calibrés empiriquement pour que la recherche se termine confortablement dans le temps imparti, quelle que soit la phase de jeu.

Temps alloué	Profondeur maximale
≤ 1 s	1
≤ 3 s	2
≤ 6 s	3
≤ 12 s	4
> 12 s	5

TABLE 2 – Temps alloué et profondeur maximale correspondante.

Le pseudo-code de la figure 9 résume le déroulement complet : on initialise d'abord un coup de secours (le premier coup légal disponible) pour ne jamais se retrouver sans réponse, puis on lance les itérations successives. À chaque profondeur complète, on met à jour le meilleur coup mémorisé. Dès qu'une itération est interrompue par la deadline, on s'arrête et on retourne le dernier coup valide enregistré.

```

fonction iterative_deepening(etat, joueur_ia, temps_max, prof_max):
    fixer une deadline
    initialiser meilleur_coup avec un coup legal

    pour profondeur croissante:
        lancer minimax a cette profondeur
        si la profondeur est entierement terminee:
            memoriser le meilleur coup
        sinon:
            arreter

    retourner meilleur_coup

```

FIGURE 9 – Pseudo-code de l'Iterative Deepening

4.3 Heuristiques

Nos algorithmes MiniMax appellent tous une fonction `evaluate_state()` qui prend en argument l'état du jeu, un id de joueur et le mode de l'IA. Nous en avons implémenté 3, 3 heuristiques basées sur différentes stratégies adaptées au jeu du quoridor.

La première est celle utilisée par défaut. Elle se concentre sur une comparaison directe entre le joueur et ses adversaires. Elle compare d'abord les distances entre l'IA et son objectif et de ses adversaires avec les leurs à partir de la fonction `get_shortest_path_length`, un algorithme de parcours en largeur (BFS) qui retourne le nombre de mouvements du chemin le plus court vers l'objectif du joueur. Si la distance de l'IA est plus faible, alors le score est positif. Elle prend également en compte le matériel en comparant son nombre de murs restants à celui de la moyenne des adversaires. Le poids accordé à la comparaison des murs est assez faible (on multiplie l'écart du nombre de murs restants par 0,5 si c'est une partie à 2 joueurs, 0,3 sinon), ce qui poussera l'IA à bouger son pion plutôt que de placer des murs.

La deuxième est basée sur le matériel, elle est plus agressive. Le poids des murs est plus élevé (x2 si 2 joueurs, 1,2 sinon) et la comparaison ne se fait pas avec la moyenne des murs restants adversaire cette fois-ci mais avec celui qui a le plus de mur uniquement. Cette différence est importante car l'IA se compare avec l'adversaire ayant le plus de murs. Ce choix permet à l'IA d'avoir une meilleure gestion de ses murs, en cas de partie où 2 adversaires n'ont plus de murs et un autre les a toujours tous, elle restera sur ses gardes plutôt que de dépenser tous ses murs car la moyenne des murs restants est basse. Également dans cette évaluation, on calcule la distance de l'adversaire le plus proche et on le multiplie par 2, l'IA va donc chercher à faire faire des détours à ses adversaires, quitte à rallonger son chemin au passage, tant qu'elle garde l'avantage du nombre de murs. Cette stratégie se concentre sur l'obstruction des adversaires, elle joue la montre, elle mise sur la victoire à long terme après l'épuisement des murs. C'est une stratégie très commune au Quoridor.

Enfin la troisième, la plus complexe, on peut l'imaginer comme une version hybride améliorée des deux premières. Elle mélange le positionnement spatial ainsi qu'une gestion dynamique des ressources. On introduit ici une composante de centralité, au quoridor, être au centre permet de plus facilement varier entre la gauche et la droite lorsqu'un adversaire pose un mur, c'est donc un avantage. Pour calculer ce bonus, on calcule la distance de Manhattan entre le pion du joueur et l'intersection entre la colonne centrale et la ligne centrale. Le poids de ce bonus est assez faible pour influencer l'IA entre 2 mouvements égaux à choisir celui qui se rapproche du centre, sans pour autant faire reculer son pion pour rester au centre, réduire la distance vers l'arrivée reste plus important. On a ensuite implémenté un poids dynamique des murs, si l'IA a plus de 2 murs, elle leur donne une faible importance (poids de 0,4). A partir de 2 murs et moins, le poids devient 0,8, elle devient plus conservatrice et préfère les conserver pour bloquer plus efficacement l'adversaire. De plus, on augmente encore plus le poids de la distance adversaire par rapport à la sienne ce qui rend l'IA plus agressive. Pour ces trois heuristiques, le calcul du chemin le plus court est coûteux mais précis, c'est donc un compromis intéressant.

Ces 3 fonctions d'évaluation sont appelées par une fonction parente, `evaluate_state`, qui est appelée par l'algorithme minimax et qui, en fonction de la valeur de l'attribut `scoring`, calcule la position avec l'heuristique choisie.

Critère	Standard (H1)	Matérielle (H2)	Hybride (H3)
Focus Principal	Distance relative	Obstruction	Positionnement
Référence Murs	Moyenne adverse	Max adverse	Moyenne (Dynamique)
Poids des Murs	0,3 - 0,5	1,2 - 2,0	0,4 → 0,8
Calcul Distance	BFS	BFS	BFS
Bonus Spécial	Aucun	Aucun	Centralité (Distance de Manhattan)

TABLE 3 – Comparaison technique des trois heuristiques implémentées.

4.4 Monte Carlo Tree Search (MCTS)

4.4.1 Introduction

Alors que l’algorithme Minimax repose sur une exploration exhaustive et déterministe à profondeur fixe, le Monte Carlo Tree Search propose une approche stochastique basée sur l’échantillonnage de l’espace des états. Dans le cadre du jeu Quoridor, le passage au MCTS est motivé par deux caractéristiques majeures du jeu :

L’explosion du facteur de branchement : Avec la possibilité de déplacer un pion ou de placer un mur parmi la multitude de coordonnées et orientations, le nombre de coups légaux par tour est élevé. Une exploration complète devient rapidement impossible à mesure que la complexité combinatoire augmente.

La difficulté d’évaluation : Contrairement aux échecs, une position dite “avantageuse” au Quoridor est difficile à quantifier par une simple heuristique positionnelle et matérielle. En effet un simple mur bien placé peut totalement inverser la distance des joueurs vers leur objectif.

Le MCTS offre une solution : il accumule des simulations pour construire statistiquement une estimation de la valeur d’un coup. En pratique, le MCTS ne cherche pas à calculer l’arbre complet du jeu, ce qui serait impossible. A chaque tour, l’algorithme lance un cycle de simulations intensives qui s’exécute de manière itérative jusqu’à l’épuisement du temps imparti (défini par l’argument `-ai-time`).

L’algorithme construit progressivement un arbre partiel en mémoire. Chaque nœud de cet arbre représente un état du plateau et stocke deux statistiques cruciales issues des simulations passées :

Le nombre de visites (N) : Combien de fois un état a été exploré.

Le nombre de victoires (W) : Le succès cumulé des joueurs à partir de ce nœud.

Plus une branche semble prometteuse, plus l’algorithme y consacre de simulations. A l’inverse, les branches menant à des défaites rapides sont délaissées. Une fois le temps écoulé, l’IA ne choisit pas nécessairement le nœud ayant le meilleur ratio de victoire, mais le nœud ayant le plus grand nombre de visites. Ce critère est le plus robuste car il désigne le coup que l’algorithme a jugé le plus digne d’intérêt après des milliers de tests.

Un cycle de simulation est divisé en 4 phases :

Sélection : En partant de la racine (l’état actuel du jeu), l’algorithme descend dans l’arbre en choisissant à chaque niveau le nœud de l’enfant le plus pertinent. Ce choix repose sur la formule UCT, cette formule permet d’équilibrer l’exploitation des coups connus comme gagnants et l’exploration de coups encore peu visités :

$$UCT = \frac{W_i}{N_i} + C \times \sqrt{\frac{\ln(N_{parent})}{N_i}}$$

Où W_i est le nombre de victoires du nœud, N_i son nombre de visites et C une constante (souvent fixée à $\sqrt{2}$, c’est le choix approximatif que nous avons fait dans notre moteur, plus précisément 1,41). Ce jonglage se fait car le premier terme (exploitation) augmente avec le taux de victoire, tandis que le second (exploration) augmente si le nœud est délaissé par rapport à ses frères.

Expansion : Lorsqu’un nœud est atteint mais qu’il possède encore des coups non explorés (`untried_moves`), l’algorithme crée un nouveau nœud enfant pour l’un de ces coups et l’ajoute à l’arbre. **Simulation (Rollout) :** Phase de plongée. A partir du nouveau nœud, l’IA simule une partie rapide jusqu’à la victoire d’un joueur. Cette simulation est habituellement aléatoire, afin de donner une idée rapide de si la position est gagnante ou non pour le joueur. Nous détaillons plus loin un choix technique concernant cette simulation afin d’améliorer notre moteur. **Rétropropagation :** Une fois le vainqueur de la simulation connu, l’algorithme remonte de nœud en nœud jusqu’à la racine pour mettre à jour les statistiques (N et W) de tous les parents traversés lors de cette itération.

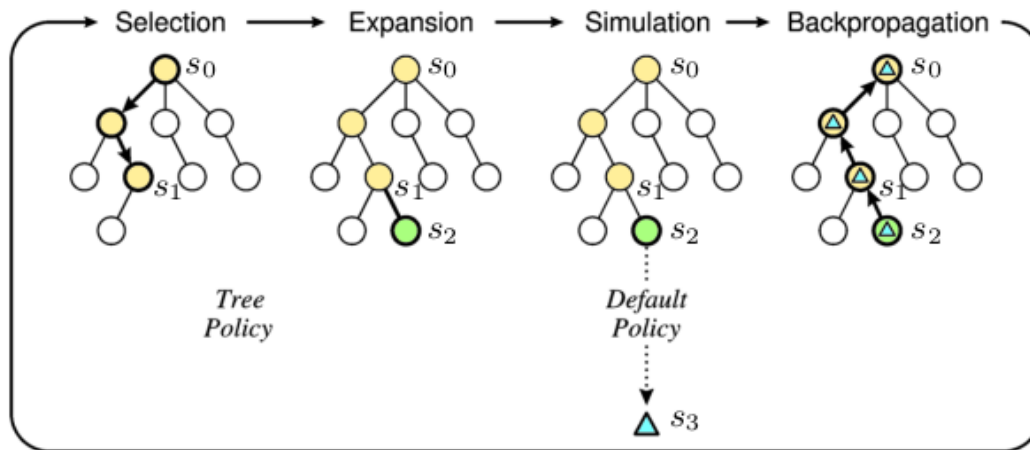


FIGURE 10 – Schéma des 4 phases d’un algorithme MCTS

Ces quatre étapes forment le cycle de base d’un algorithme MCTS, dont l’efficacité repose théoriquement sur la convergence statistique vers les meilleurs coups au fil des simulations. Ce modèle est robuste pour des jeux à faible complexité mais présente de réelles limites lorsqu’on s’attaque à des jeux plus vastes comme le Quoridor.

4.4.2 Problématique

En effet, l’application d’un MCTS “pur”, donc totalement aléatoire dans la simulation des coups, au Quoridor s’est avérée très instable lors de nos premiers tests. En effet, le facteur de branchement est tellement grand que simuler des coups aléatoires jusqu’à la victoire de l’un des joueurs à chaque nœud va créer un arbre beaucoup trop large et l’algorithme n’a pas le temps d’explorer suffisamment pour trouver un coup intéressant en seulement quelques secondes. Les simulations produisent un “bruit” important car les pions tendent à errer sans but et à placer des murs inutiles, ce qui rend les statistiques de victoire peu fiables pour l’algorithme.

Prenons pour exemple une partie à 2 joueurs sur un plateau de 9x9 cases. dès le premier coup, il y a 3 mouvements de pions possibles (avancer, droite, gauche) et 128 murs possibles (8 murs différents par ligne / colonne, donc 64 par orientations), on a un total de 131 coups légaux différents. Au mieux, les coups suivants auront minimum 124 possibilités et ainsi de suite, l’arbre devient très vite trop large et sa complexité augmente trop vite. Un algorithme explorant cet arbre de manière aléatoire va très vite se perdre.

Pour pallier ces limites, notre implémentation s’écarte du modèle classique en introduisant un élagage sélectif des coups, plus particulièrement sur le placement des murs, ainsi que des simulations guidées par des heuristiques de distance. Cette approche permet de concentrer la puissance de calcul sur les séquences de jeu les plus crédibles, réduisant ainsi drastiquement la largeur de l’arbre au profit de sa profondeur.

4.4.3 Optimisation de l’espace de recherche / Architecture et fonctionnement

L’efficacité de notre moteur MCTS (dans le fichier `mcts_engine.py`) repose sur une rupture avec l’approche “brute” classique. Au lieu d’explorer l’intégralité des coups légaux, l’algorithme se concentre sur un sous-ensemble de mouvements qualitatifs, réduisant ainsi le facteur de branchement de manière drastique.

Élagage stratégique des coups La fonction `get_tree_moves` est le premier filtre de l’algorithme. Dans une partie de Quoridor, nous avons vu que le nombre de murs posables est immense alors que la majorité n’ont aucun impact stratégique immédiat. A la place donc de créer un nœud représentant un état du jeu avec une liste de tous les coups possible depuis cet état, nous filtrons cette liste en nous basant sur 3 points importants :

Priorité aux déplacements : L’algorithme inclut systématiquement tous les déplacements de pions légaux.

Ciblage des murs : Nous utilisons la fonction `get_blocking_walls`, une heuristique de filtrage qui va tester à l’aide d’un algorithme BFS tous les murs et ne garder seulement les murs dits “bloquants”. Un mur est considéré comme tel s’il augmente la distance du chemin le plus court de l’adversaire vers sa ligne d’arrivée. Nous effectuons un `clone_state(state)` à chaque itération, c’est l’opération la plus lourde de cette fonction mais l’efficacité du filtrage justifie ce coût de clonage répété.

Activation tardive : L’exploration intensive des murs n’est activée que lorsque l’adversaire est à une distance inférieure à 5 de son objectif (on calcul ici la distance axiale, sans prendre en compte les murs, de la ligne / colonne du joueur jusqu’à sa ligne / colonne d’arrivée). Cela permet à l’IA de se concentrer sur sa propre progression en début de partie tout en devenant extrêmement défensive par la suite.

Il est important de préciser que lorsque la fonction `get_blocking_walls` retourne la liste des murs bloquant, la fonction `get_tree_move` n'en sélectionne que 3 au hasard. L'objectif ici est d'éviter l'explosion combinatoire, en effet même en ne gardant que les murs bloquants, il peut y en avoir une dizaine à chaque coups et le problème n'aurait été que déplacé. En ne prenant que 3 murs, nous faisons le pari statistique que parmi ces 3, au moins un sera suffisamment pertinent pour contrer l'adversaire. Étant donné que le MCTS est lancé plusieurs fois, l'aléatoire permet de tester des murs différents d'une partie à l'autre, finissant par identifier ceux qui mènent à la victoire sans que les options de murs ne saturent totalement le temps de calcul.

Simulations guidées par heuristiques Le point critique d'un MCTS est la phase de Simulation. Si les deux joueurs jouent n'importe comment, le résultat de la simulation ne reflète pas la qualité réelle du coup initial. Pour résoudre ce problème, nous avons implémenté une politique de simulation guidée.

- **Biais de progression** : Lors d'un rollout, le joueur n'est pas aléatoire. Il y a une forte probabilité (85%) qu'il choisisse un coup réduisant sa distance vers l'objectif (calculé par `get_shortest_path_length`). En forçant ainsi les pions à se diriger vers leur but, les simulations se terminent plus rapidement et imitent un comportement humain. Les statistiques de victoire (win / visits) deviennent alors un indicateur fiable de la solidité d'une position.
- **Gestion des murs en simulation** : Les rollouts ne posent jamais de murs, ils simulent uniquement la fin de partie avec les pions. Cela fait gagner en rapidité de calcul durant les simulations mais poussera l'IA à beaucoup consommer ses murs car elle voit l'avantage immédiat d'un mur posé dans l'arbre, mais elle ne va pas simuler le fait que l'adversaire pourrait répondre par un autre mur.
- **Compromis qualité/quantité** : Nous avons conscience qu'utiliser un algorithme de parcours en largeur pour calculer la distance la plus courte à chaque rollout est coûteux, mais ce coût est largement compensé par la pertinence des résultats obtenus. Nous choisissons en quelque sorte la qualité à la quantité, nous perdons en vitesse brute mais gagnons en précision.
- **Robustesse** : Nous avons fait le choix de garder 15% d'aléatoire ici afin de préserver la nature explorative du MCTS. Un rollout purement dirigé par le plus court chemin rendrait l'évaluation trop rigide et incapable d'anticiper les comportements humain de l'adversaire. Ce mélange permet d'obtenir des simulations qui sont à la fois efficaces et diversifiées.

Ainsi, nous avons conçu un MCTS capable de rivaliser avec un Minimax. Là où un MCTS classique échouerait par manque de temps et trop de branches à visiter, le nôtre va à chaque fois retourner un coup utile. Tout cela en fait un réel adversaire, imparfait évidemment, mais suffisamment intelligent pour offrir au joueur une partie vraiment intéressante.

4.5 Pipeline d'apprentissage automatique

L'idée de départ est simple : plutôt que d'utiliser la formule UCT classique pour guider le MCTS vers les nœuds les plus prometteurs, pourquoi ne pas entraîner un modèle de machine learning à reconnaître les bons coups ? L'objectif n'est pas de remplacer tout le moteur de recherche, mais uniquement la politique de sélection : à chaque nœud, c'est le modèle qui décide quel enfant explorer en priorité, en lui attribuant une probabilité de victoire estimée.

Pour y arriver, nous avons suivi un pipeline en quatre étapes : générer des parties de référence, en extraire des données structurées, entraîner un modèle, puis l'intégrer dans le MCTS. La figure 10 illustre ce déroulement.

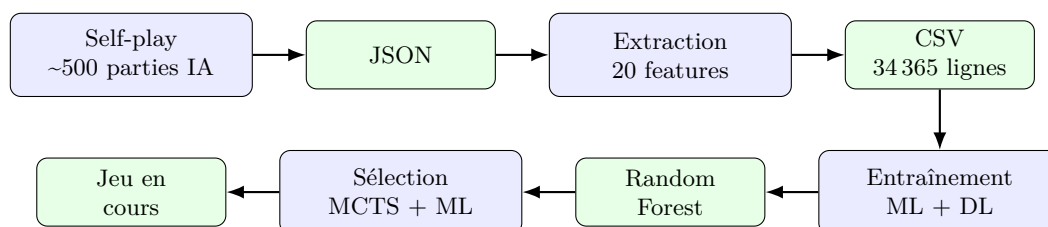


FIGURE 11 – Pipeline ML : génération, entraînement et intégration en jeu

Génération des données. La première étape consistait à produire des parties de bonne qualité pour constituer notre jeu de données. Nous avons développé un fichier de génération dédié (`generate_games.py`) qui contient à la fois la logique de lancement des parties et une classe permettant d'instancier n'importe quel type de bot. Grâce à cette classe, nous avons pu créer des bots couvrant l'ensemble de nos IA : Minimax et Iterative Deepening avec les trois heuristiques (défaut, matériel, hybride) et différents paramétrages de profondeur et de temps, ainsi que le MCTS pur. Ces bots jouent ensuite les uns contre les autres en self-play.

Dans un premier temps, nous avons généré 80 parties. Le modèle entraîné sur ce volume était trop pauvre : il n'avait pas vu suffisamment de situations de jeu différentes pour apprendre à distinguer un bon coup d'un mauvais. Nous avons donc augmenté progressivement jusqu'à atteindre environ **500 parties**. Chaque partie est sauvegardée au format JSON et contient toutes les informations nécessaires à la reconstruction de la partie : les coups joués tour par tour, les états successifs du plateau, et le joueur vainqueur.

Extraction des features. Une fois les parties générées, nous avons rejoué chaque partie coup par coup pour en extraire un vecteur de 20 features par mouvement. Ces features sont organisées en trois groupes qui décrivent la situation avant, pendant et après chaque coup.

Contexte avant le coup (13 features). Ce groupe capture la situation globale de la partie au moment où le coup est joué. On y trouve par exemple la phase de jeu (`game_phase`, qui distingue l'ouverture de la fin de partie), les distances de chaque joueur vers son objectif, l'avantage de distance entre l'IA et ses adversaires, le nombre de murs restants pour chacun, et les positions actuelles des pions sur le plateau. Ces informations donnent au modèle une vision d'ensemble du contexte stratégique.

Description du coup (5 features). Ce groupe caractérise la nature du coup joué : s'agit-il d'un déplacement de pion ou d'un placement de mur ? Si c'est un mur, est-il horizontal ou vertical ? Si c'est un déplacement, quelle distance parcourt le pion ? Par exemple, `is_wall_move` vaut 1 si le coup est un mur et 0 sinon, ce qui permet au modèle d'apprendre des stratégies différentes selon le type d'action.

Effet du coup (2 features). Ce groupe mesure les conséquences immédiates du coup : `my_progress` quantifie le gain de proximité vers l'objectif pour l'IA, et `opp_slowdown` mesure le ralentissement infligé à l'adversaire. Ce sont les deux features les plus directement liées à la qualité d'un coup, et elles s'avèrent parmi les plus importantes pour le modèle.

La figure 11 résume la décomposition complète des 20 features.

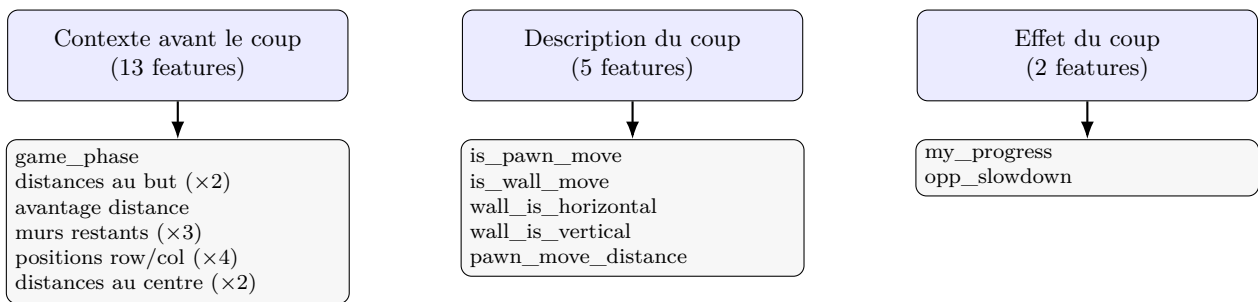


FIGURE 12 – Les 20 features utilisées pour la sélection ML

Au terme de cette extraction, nous obtenons un fichier CSV de **34 365 lignes**, où chaque ligne correspond à un coup joué lors d'une des 500 parties générées.

Entraînement et sélection du modèle. L'entraînement est réalisé dans le notebook ML/data/Training.ipynb. Les données sont séparées en 80 % pour l'apprentissage et 20 % pour le test, avec stratification pour conserver la distribution des classes. Nous avons comparé quatre modèles classiques de scikit-learn, dont les résultats sont présentés dans le tableau 4.

Nous évaluons chaque modèle sur quatre métriques : **Train Acc**, le taux de bons coups sur les données d'entraînement ; **Test Acc**, le même taux sur des données jamais vues, qui reflète la vraie capacité du modèle ; **F1**, qui équilibre précision et rappel ; et **AUC**, qui mesure à quel point le modèle sépare bien les bons coups des mauvais (1,0 étant parfait, 0,5 étant aléatoire).

Modèle	Train Acc	Test Acc	F1	AUC
Régression Logistique	0,757	0,755	0,759	0,833
Arbre de Décision ($d=5$)	0,840	0,836	0,842	0,931
Random Forest (100)	1,000	0,938	0,939	0,987
Gradient Boosting	0,869	0,866	0,867	0,946

TABLE 4 – Comparaison des modèles ML

La **Régression Logistique** sert de référence : avec une AUC de 0,833 et une accuracy de 75,5 %, elle apprend les tendances générales mais reste trop linéaire pour capturer la complexité du jeu. L'**Arbre de Décision** progresse nettement (AUC 0,931, accuracy 83,6 %), mais sa profondeur limitée à 5 lui interdit de modéliser des interactions fines

entre features. Le **Gradient Boosting** fait encore mieux (AUC 0,946, accuracy 86,6%), mais au prix d'un temps d'inférence plus élevé. Le **Random Forest** (100 arbres) domine sur tous les critères : AUC de 0,987, accuracy de 93,8% et F1 de 0,939. Malgré un score d'entraînement à 1,000 qui indique un surajustement sur les données d'entraînement, ses performances sur le jeu de test restent très solides. C'est donc ce modèle que nous avons retenu.

Comparaison avec un réseau de neurones. Pour vérifier que notre choix du Random Forest était le plus pertinent, nous avons également développé un réseau de neurones (MLP, *Multilayer Perceptron*) avec Keras. L'idée d'un MLP est simple : les 20 features d'un coup traversent plusieurs couches de calcul successives, et chaque couche apprend à combiner les informations de la couche précédente pour en extraire des patterns de plus en plus abstraits, jusqu'à produire une estimation finale.

Notre réseau est composé de trois couches denses successives de 128, 64 puis 32 neurones — le nombre diminue progressivement pour forcer le réseau à condenser l'information. Après chaque couche, on applique une **normalisation par batch** (qui stabilise les valeurs entre couches pour accélérer l'apprentissage) et un **dropout** (qui désactive aléatoirement une partie des neurones pendant l'entraînement, forçant le réseau à ne pas dépendre d'un seul chemin de calcul et évitant ainsi de trop coller aux données d'entraînement). Une dernière couche à un neurone avec activation **sigmoïde** — une fonction qui écrase n'importe quelle valeur entre 0 et 1 — produit directement la probabilité de victoire estimée. La figure 5 présente l'architecture complète.

Layer (type)	Output Shape	Param #
dense	(None, 128)	2 688
batch_normalization	(None, 128)	512
dropout	(None, 128)	0
dense_1	(None, 64)	8 256
batch_normalization_1	(None, 64)	256
dropout_1	(None, 64)	0
dense_2	(None, 32)	2 080
batch_normalization_2	(None, 32)	128
dropout_2	(None, 32)	0
dense_3	(None, 1)	33

TABLE 5 – Architecture du MLP utilisé pour comparaison

La figure 12 compare les résultats du MLP et du Random Forest sur le jeu de test. Le MLP atteint une AUC de 0,975 et une accuracy de 89,9%, ce qui reste honnête, mais le Random Forest le surpasse sur tous les indicateurs. Nous avons donc conservé le Random Forest : il est plus précis, plus rapide à l'inférence, et plus simple à intégrer dans le moteur de jeu sans dépendance à Keras.

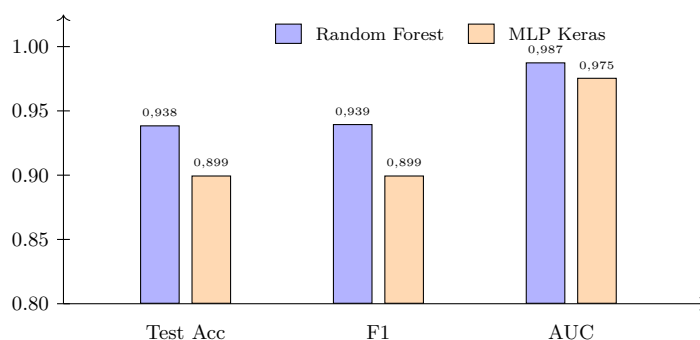


FIGURE 13 – Comparaison Random Forest / MLP sur le jeu de test

Intégration dans le MCTS. La sélection ML remplace uniquement la formule UCT. Pour chaque enfant d'un nœud, le programme reconstruit les 20 features à partir de l'état avant le coup, du coup lui-même et de l'état après le coup. Le modèle calcule ensuite une probabilité de victoire, et l'enfant retenu est celui dont cette probabilité est maximale. Les autres étapes du MCTS restent inchangées : expansion, rollout et rétro-propagation.

Le mode est activé via `-ai-mcts-selection ML`. Le modèle est chargé une seule fois puis mis en cache. Le pseudo-code est donné en figure 13. Il représente uniquement l'étape de sélection : pour chaque enfant déjà généré, on calcule un score ML, puis on choisit celui qui obtient la meilleure probabilité de victoire.

```
fonction ml_select_child(noeud_parent):
    pour chaque enfant du noeud:
        construire ses features
        estimer sa probabilité de victoire avec le modele

    retourner l'enfant avec le meilleur score
```

FIGURE 14 – Pseudo-code de la sélection ML dans le MCTS

5 Conception et Implémentation de l'Interface Graphique

Le projet Quoridor nécessite une interface graphique permettant à l'utilisateur de jouer avec sa souris : voir le plateau, déplacer ses pions, poser des murs, et accéder aux fonctionnalités du programme comme l'intelligence artificielle, le mode réseau ou le chronomètre. L'interface est implémentée à l'aide de **GTK4**, un ensemble d'outils permettant de créer des fenêtres et des éléments visuels en Python. Elle constitue uniquement la couche de présentation : elle n'implémente aucune règle du jeu, aucun algorithme d'intelligence artificielle, aucune logique réseau. Pour chacune de ces fonctionnalités, elle délègue à des modules déjà implémentés (`GameSession`, `GameApplicationService`, `Blitz`, `NetworkClient`), et se contente d'afficher l'état du jeu à l'écran et de transmettre les actions de l'utilisateur à ces modules. Cette séparation stricte entre présentation et logique garantit que la CLI et le GUI partagent exactement le même cœur de jeu.

5.1 Dessiner le plateau et interagir avec lui

Pour afficher le plateau, l'approche la plus intuitive consisterait à placer une grille de 81 boutons cliquables un par case. GTK propose nativement des widgets `Gtk.Button` qui gèrent automatiquement les clics. Cette solution se heurte cependant à trois obstacles rédhibitoires pour Quoridor. Les murs ne se posent pas *sur* les cases mais *entre* elles : une grille de boutons n'offre aucun espace naturel pour les représenter, ce qui imposerait plus de 200 widgets supplémentaires avec une logique de coordonnées complexe. De plus, l'apparence des boutons GTK dépend du thème graphique du système d'exploitation, rendant impossible un rendu visuel homogène et contrôlé. Enfin, le glisser-déposer du pion nécessite de suivre la position de la souris en temps réel, ce qu'un simple bouton cliquable ne sait pas faire.

Pour contourner ces limites, on remplace les 81 boutons par un **canvas de dessin** : un widget `Gtk.DrawingArea`, une zone rectangulaire vierge sur laquelle on dessine tout le plateau manuellement à l'aide de **Cairo**, une bibliothèque de dessin vectoriel. Cairo permet de décrire des formes géométriques (rectangles, cercles, lignes) indépendamment de la résolution de l'écran, ce qui garantit un rendu net quelle que soit la taille de la fenêtre.

Extrait 1 – Création de la zone de dessin

```
1 total = SIZE * CELL + (SIZE - 1) * GAP + 2 * MARGIN
2 self.area = Gtk.DrawingArea(hexexpand=True, vexpand=True)
3 self.area.set_content_width(total)
4 self.area.set_content_height(total)
5 self.area.set_draw_func(self._draw)
```

Une unique méthode `_draw` dessine l'intégralité du plateau : d'abord le fond, puis les cases, puis les murs dans leurs interstices, et enfin les pions par-dessus. L'ordre est important Cairo dessine comme un peintre, chaque élément recouvrant ceux dessinés avant lui. À chaque changement d'état du jeu, un appel à `self.area.queue_draw()` demande à GTK de redéclencher ce redessin. La taille d'une case est recalculée dynamiquement à chaque redessin par `_cell_size` :

Extrait 2 – Calcul dynamique de la taille d'une case

```
1 def _cell_size(self):
2     size = self._board_size()
3     available = (
4         min(self.area.get_width(), self.area.get_height()) - 2 * MARGIN
5     )
6     return max((available - (size - 1) * GAP) / size, 1)
```

On prend le plus petit côté de la fenêtre (`min(width, height)`) pour garantir que le plateau reste toujours carré même si l'utilisateur étire la fenêtre horizontalement. On soustrait ensuite les marges (`2 * MARGIN`) puis l'espace total

occupé par les interstices entre les cases $((\text{size} - 1) * \text{GAP})$ il y a $\text{size} - 1$ espaces pour size cases. Ce qui reste est divisé par le nombre de cases pour obtenir la taille d'une case. Le `max(..., 1)` est une sécurité qui évite de retourner zéro ou une valeur négative si la fenêtre devient trop petite, ce qui ferait planter Cairo.

La méthode `_xy_to_cell` effectue la transformation inverse : à partir des coordonnées en pixels d'un clic, elle retrouve la case correspondante.

Extrait 3 – Conversion d'un pixel en coordonnée de case

```

1 def _xy_to_cell(self, x, y):
2     size = self._board_size()
3     cs = self._cell_size()
4     step = cs + GAP
5     col = int((x - self._ox) / step)
6     row = int((y - self._oy) / step)
7     if 0 <= row < size and 0 <= col < size:
8         in_cell_x = (x - self._ox) - col * step
9         in_cell_y = (y - self._oy) - row * step
10        if in_cell_x <= cs and in_cell_y <= cs:
11            return row, col
12    return None

```

La division par `step` donne le numéro de colonne dans lequel tombe le pixel `x`. Mais cela ne suffit pas : si le clic tombe dans l'interstice entre deux cases, la division donnerait quand même un numéro de colonne valide. Le second test vérifier que `in_cell_x` et `in_cell_y` ne dépassent pas `cs` permet de détecter ce cas et de retourner `None`, signifiant qu'aucune case n'a été cliquée. Ce filtre est essentiel pour distinguer un clic sur une case d'un clic dans un interstice destiné à poser un mur.



FIGURE 15 – Interface graphique du jeu Quoridor en cours de partie. Les murs s'affichent dans les interstices entre les cases, rendus par Cairo via un unique widget `Gtk.DrawingArea`.

5.2 Interagir avec le plateau : le glisser-déposer

Une fois le plateau affiché, l'utilisateur doit pouvoir déplacer son pion et poser des murs avec la souris. GTK4 propose le concept d'**EventController**, un objet qui capture des gestes de haut niveau plutôt que des événements bruts de la souris. On utilise `Gtk.GestureDrag`, qui émet trois signaux correspondant aux trois phases du geste : `drag-begin` au moment où l'utilisateur appuie sur le bouton de la souris, `drag-update` pendant le déplacement, et `drag-end` au relâchement. Cette approche est plus robuste que l'écoute directe des événements bas niveau car le geste gère automatiquement les cas particuliers comme la perte de focus ou l'annulation.

Phase 1 : Détecter l'intention

La difficulté propre à Quoridor est que le même geste peut exprimer deux intentions différentes selon l'endroit où il commence. Au moment du **drag-begin**, l'interface doit donc trancher : si le clic part de la case occupée par le pion du joueur courant, c'est un déplacement de pion ; si le clic part d'un interstice entre deux cases, c'est un placement de mur. Cette distinction est réalisée en deux temps. D'abord `_xy_to_cell` vérifie si le clic tombe sur une case. Si ce n'est pas le cas, `_xy_to_gap` prend le relais et analyse si le curseur se trouve dans un interstice :

Extrait 4 – Détection d'un interstice entre deux cases

```
1 def _xy_to_gap(self, x, y):
2     cs = self._cell_size()
3     step = cs + GAP
4     col_idx = int(rel_x / step)
5     row_idx = int(rel_y / step)
6     in_cell_x = rel_x - col_idx * step
7     in_cell_y = rel_y - row_idx * step
8     if in_cell_x >= cs and not in_cell_y >= cs:
9         return ("vertical", row_idx, col_idx)
10    if in_cell_y >= cs and not in_cell_x >= cs:
11        return ("horizontal", row_idx, col_idx)
12    return None
```

Le raisonnement est géométrique : si la position locale dans le pas (`in_cell_x`) dépasse la taille d'une case (`cs`), le curseur est dans la zone de séparation à droite de la case, donc dans un interstice vertical. Le même raisonnement s'applique verticalement pour détecter un interstice horizontal. Si ni l'un ni l'autre, le clic ne correspond à aucune action valide et est ignoré. Une fois l'interstice identifié, `_gap_to_wall_edges` traduit cette position visuelle en paires de nœuds du graphe de jeu, puis le module chargé de la vérification des règles du jeu vérifie que l'action est légale

Phase 2 : Animer le déplacement

Pendant le **drag-update**, aucun coup n'est encore appliqué. L'interface se contente de mémoriser le décalage courant de la souris par rapport au point de départ et de demander un redessin :

Extrait 5 – Suivi visuel du pion pendant le glissement

```
1 def _on_drag_update(self, gesture, offset_x, offset_y):
2     if self._drag_pid is None:
3         return
4     self._drag_offset = (offset_x, offset_y)
5     self.area.queue_draw()
```

La méthode `_draw_players` utilise cet offset pour dessiner le pion à la position courante de la souris plutôt qu'au centre de sa case d'origine, donnant l'impression que le pion suit le curseur en temps réel. C'est purement visuel : l'état du jeu n'est pas modifié tant que le joueur n'a pas relâché le bouton.

Phase 3 : Valider et appliquer le coup

Au **drag-end**, l'interface calcule la case cible à partir de la position finale du curseur, puis appelle `validate_pawn_move` pour vérifier que le déplacement est légal adjacence, sauts par-dessus un adversaire, sauts diagonaux en cas de mur bloquant. Si le coup est valide, il est appliqué à la session de jeu et le plateau est redessiné. Si le coup est invalide, un message est affiché 15 dans la barre de statut et le pion revient à sa position d'origine. Dans tous les cas, l'état du drag est réinitialisé en fin de méthode pour éviter qu'un drag échoué ne laisse le pion collé à la souris lors du prochain geste.

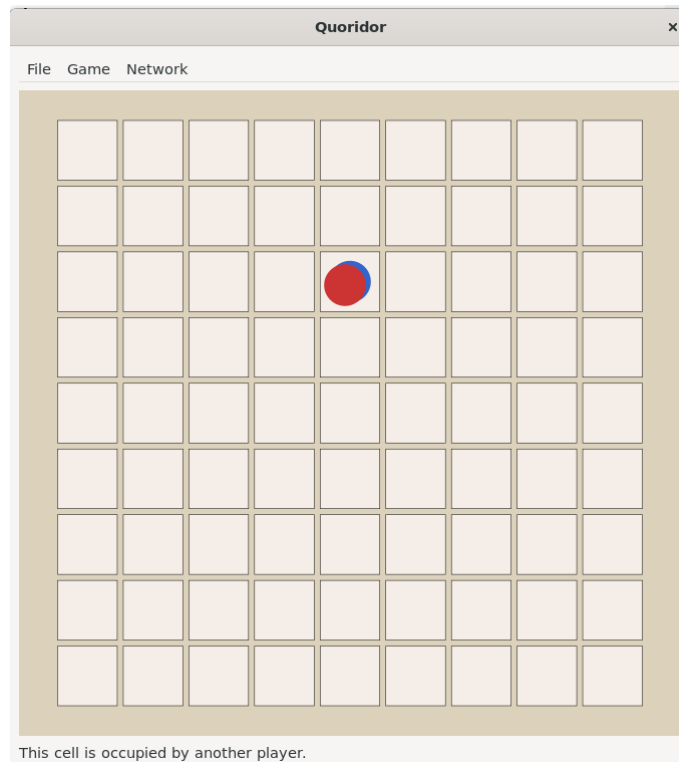


FIGURE 16 – Tentative de déplacement illégal : le joueur tente de se déplacer sur une case occupée par l’adversaire, la barre de statut affiche un message d’erreur explicite.

5.3 Gérer les menus et les raccourcis clavier

L’interface expose dix-neuf actions à l’utilisateur — nouvelle partie, chargement, sauvegarde, annuler, rejouer, pause, indice, rejoindre un serveur, démarrer un serveur, etc. — accessibles depuis trois menus (File, Game, Network) et des raccourcis clavier. Ces raccourcis doivent de plus être reconfigurables par l’utilisateur et sauvegardés entre les sessions.

La manière la plus directe de relier un élément de menu à une action serait de connecter chaque bouton du menu directement à la fonction Python qui réalise l’action. Avec dix-neuf actions pouvant chacune être déclenchées par un élément de menu *et* un raccourci clavier, cette approche crée rapidement un enchevêtrement de connexions : si l’on veut modifier une action, il faut retrouver et modifier tous les endroits du code où elle est connectée. Pour éviter ce problème, on introduit un intermédiaire.

Chaque action est d’abord identifiée par un nom unique grâce au mécanisme `Gio.SimpleAction` de GTK. Ce nom joue le rôle d’étiquette : le menu et le raccourci clavier référencent tous les deux cette étiquette, sans connaître la fonction réelle qui sera exécutée. C’est seulement à un endroit centralisé — la méthode `_install_actions` — que l’on fait le lien entre chaque étiquette et sa fonction :

Extrait 6 – Enregistrement unifié des actions

```

1 def _install_actions(self) -> None:
2     handlers = {
3         ActionType.NEW_GAME: self._action_new_game,
4         ActionType.UNDO:     self._action_undo,
5         ActionType.REDO:     self._action_redo,
6         ActionType.PAUSE:    self._action_pause,
7         # ... dix-neuf actions au total ...
8     }
9     for action_type, callback in handlers.items():
10        self.action_registry.register(action_type, callback)
11        action = Gio.SimpleAction.new(action_type.value, None)
12        action.connect("activate", self._on_action_activated, action_type)
13        self.add_action(action)

```

La table `handlers` est un dictionnaire qui associe chaque action à sa fonction. La boucle parcourt ce dictionnaire et enregistre chaque association dans l’`ActionRegistry` — une classe dont le seul rôle est de mémoriser ces associations

et de les exécuter à la demande. Ajouter une nouvelle action au programme ne demande alors qu’une seule ligne dans ce dictionnaire : le menu, le raccourci clavier et la gestion d’erreurs sont automatiquement pris en charge.

Ce point d’entrée unique apporte un second avantage : la gestion des erreurs. Lorsqu’une action est déclenchée, qu’elle vienne du menu ou d’un raccourci, elle passe toujours par la même fonction `_on_action_activated`. Si quelque chose se passe mal — un fichier introuvable, un coup illégal, une perte de connexion — l’erreur est interceptée à cet endroit et son message est affiché dans la barre de statut en bas de la fenêtre, sans jamais faire planter l’interface.

Enfin, pour la personnalisation des raccourcis, l’utilisateur peut modifier chaque raccourci depuis un menu dédié. Ces modifications sont sauvegardées dans un fichier de configuration `.qoridorrc` placé dans le dossier personnel de l’utilisateur, et rechargées automatiquement au prochain lancement du programme. Avant toute sauvegarde, une vérification s’assure qu’aucun raccourci n’est assigné à deux actions différentes en même temps, ce qui provoquerait des conflits.

5.4 Maintenir la fluidité face aux calculs longs

Le programme offre la possibilité de jouer contre une intelligence artificielle. Pour déterminer le meilleur coup à jouer, celle-ci explore un grand nombre de positions possibles, ce qui peut prendre plusieurs secondes. Or GTK repose sur une **boucle d’événements** : un mécanisme qui attend qu’un événement se produise (clic, frappe au clavier, demande de redessin), appelle la fonction correspondante, puis reprend son attente. Cette boucle est à *thread unique* : tant qu’une fonction ne lui a pas rendu la main, elle est bloquée et ne peut rien faire d’autre. Si l’IA calcule directement dans cette boucle, l’interface se fige complètement pendant toute la durée du calcul — l’utilisateur voit la fenêtre grisée et croit que le programme a planté.

La solution naturelle est de déplacer ces calculs dans un **thread** séparé, c’est-à-dire un fil d’exécution parallèle permettant de faire tourner plusieurs tâches en même temps pour ne pas bloquer la boucle principale. Cependant, GTK n’est pas conçu pour être manipulé depuis plusieurs threads simultanément : appeler une fonction GTK depuis un thread secondaire provoque des comportements imprévisibles, voire une corruption de la mémoire. Il faut donc calculer dans un thread séparé, puis revenir dans le thread principal pour mettre à jour l’interface.

Le calcul de l’IA est lancé dans un thread marqué `daemon=True`. Ce marquage est essentiel : sans lui, si l’utilisateur ferme la fenêtre pendant que l’IA réfléchit, Python attendrait la fin du calcul avant de fermer le processus, le laissant tourner en arrière-plan sans que l’utilisateur ne le sache.

Extrait 7 – Lancement du calcul IA dans un thread séparé

```
1 thread = threading.Thread(  
2     target=self._compute_ai_move_thread,  
3     args=(current,),  
4     daemon=True,  
5 )  
6 thread.start()
```

Pour revenir dans le thread principal après le calcul, GTK fournit `GLib.idle_add` : cette fonction place un appel dans la file d’attente de la boucle principale, qui l’exécutera dès qu’elle sera libre. Le thread de calcul ne touche jamais à GTK directement il se contente de déposer son résultat dans cette file :

Extrait 8 – Retour dans le thread principal via `idle_add`

```
1 # Thread IA  
2 def _compute_ai_move_thread(self, current: int) -> None:  
3     move = self.session.compute_ai_move(...)  
4     GLib.idle_add(self._apply_ai_move_gtk, current, move, elapsed)
```

5.5 Mesurer le temps avec précision en mode Blitz

Le mode Blitz impose une limite de temps par joueur : l’interface doit afficher le temps restant et déclarer perdant le joueur dont le chronomètre atteint zéro 16. GTK fournit `GLib.timeout_add`, une fonction qui appelle périodiquement une autre fonction à intervalle régulier. L’approche naïve consisterait à incrémenter un compteur de 100 millisecondes à chaque appel. Cependant, `GLib.timeout_add` est un *ordonnanceur*, pas une *horloge* : il garantit que l’appel n’aura pas lieu *avant* l’intervalle demandé, mais il peut arriver bien plus tard si la boucle est occupée ou si le système suspend brièvement le programme. Cumuler des incréments fixes produirait une horloge systématiquement en retard un joueur gagnerait ainsi du temps réel simplement parce que la machine est chargée.

Avant de résoudre ce problème, notons que l’objet **Blitz** est instancié même quand le mode est désactivé, avec `time_limit_minutes=0`. Sa méthode `is_enabled()` retourne alors `False` et ses autres méthodes ne font rien. C’est le **pattern Null Object** : plutôt que de parsemer le code de vérifications `if self.blitz is None`, on garantit que l’objet existe toujours avec la même interface, qu’il soit actif ou non, simplifiant considérablement le code appelant.

Pour la mesure du temps, la solution consiste à séparer complètement la cadence de rafraîchissement de l’affichage et la mesure du temps réel écoulé. Au début de chaque tour, on mémorise l’instant précis de démarrage via `time.time()`, qui lit directement l’horloge du système :

Extrait 9 – Démarrage du chronomètre

```
1 def _start_blitz_turn(self) -> None:
2     if not self.blitz.is_enabled():
3         return
4     self._turn_start_time = time.time()
5     self._blitz_timer_id = GLib.timeout_add(100, self._on_blitz_tick)
```

À chaque appel du timer, le temps écoulé est recalculé entièrement depuis cet instant de démarrage, sans jamais accumuler d’incréments :

Extrait 10 – Calcul du temps sans dérive

```
1 def _on_blitz_tick(self) -> bool:
2     if self._paused:
3         return True
4     elapsed = time.time() - self._turn_start_time
5     remaining = self.blitz.remaining_time(player_id) - elapsed
6     if remaining <= 0:
7         self._stop_blitz_turn(player_id)
8         self.session.timeout_player(player_id)
9     # ... mise a jour de l’affichage ...
```

Si un appel du timer arrive en retard, le calcul suivant lira l’heure réelle du système et verra le temps effectivement écoulé : aucune dérive n’est possible. `GLib.timeout_add` n’est plus qu’un moyen de rafraîchir l’affichage à une cadence raisonnable. Le même principe s’applique à la pause : on mémorise le temps écoulé au moment de la suspension, puis on recule `_turn_start_time` à la reprise pour que le calcul reste immédiatement correct.

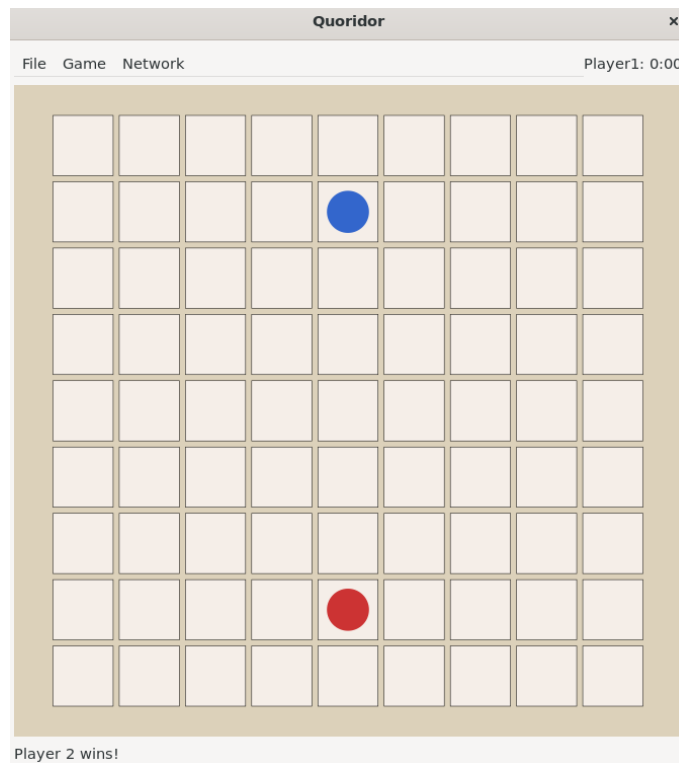


FIGURE 17 – Mode Blitz : le temps du joueur 1 est écoulé (Player1: 0:00), le joueur 2 est déclaré vainqueur par timeout.

5.6 Intégrer le mode réseau dans l’interface

Le programme permet de jouer en réseau contre un adversaire distant. Dans ce mode, le GUI ne pilote plus le jeu localement : c’est le serveur qui valide chaque coup et renvoie le nouvel état de la partie aux deux clients. L’interface

doit donc abandonner son rôle de décideur pour devenir un afficheur d'état distant, tout en continuant à répondre aux interactions de l'utilisateur.

La connexion au serveur est initiée depuis un formulaire collectant l'adresse du serveur, le port et le pseudonyme du joueur. Une fois connecté, quatre réactions sont enregistrées auprès du module réseau : une pour recevoir le nouvel état du jeu après chaque coup, une pour recevoir les notifications comme les invitations, une pour détecter les déconnexions, et une pour recevoir les coups adverses. Chacune est enveloppée dans un `GLib.idle_add`, exactement comme pour l'IA, car le module réseau s'exécute dans son propre thread de réception et ne peut pas toucher à GTK directement.

Lorsque le serveur renvoie un nouvel état après un coup valide, l'interface restaure l'état complet reçu plutôt que d'appliquer seulement le coup adverse :

Extrait 11 – Restauration de l'état reçu du serveur

```
1 def _apply_game_state_update(self, update) -> None:
2     before_state = self.session.state.to_snapshot()
3     self.session.state.restore(update["state"])
4     record = MoveRecord(
5         player_id=player_id, player_type="human",
6         action="move_pawn",
7         before_state=before_state,
8         after_state=update["state"],
9     )
10    self.session.history.record_move(record)
11    self.area.queue_draw()
12    if update["winner_id"]:
13        self._game_over = True
```

Ce choix est délibéré : si un message réseau est perdu ou si le client se désynchronise momentanément, recevoir l'état complet garantit qu'on ne diverge jamais du serveur. L'état d'une partie de Quoridor étant minuscule, ce choix de robustesse n'a aucun coût significatif. Un enregistrement du coup est ensuite reconstruit manuellement pour maintenir la cohérence de l'historique local.

Le mode réseau impose également des restrictions sur certaines fonctionnalités : l'annulation et la pause sont désactivées, car les appliquer unilatéralement créerait une désynchronisation entre les deux clients. Lorsqu'une invitation est reçue comme illustré en figure 17, l'interface affiche une fenêtre modale permettant de l'accepter ou de la refuser sans bloquer la boucle d'événements. En cas de déconnexion inattendue, l'interface repasse automatiquement en mode local sans laisser le programme dans un état incohérent.

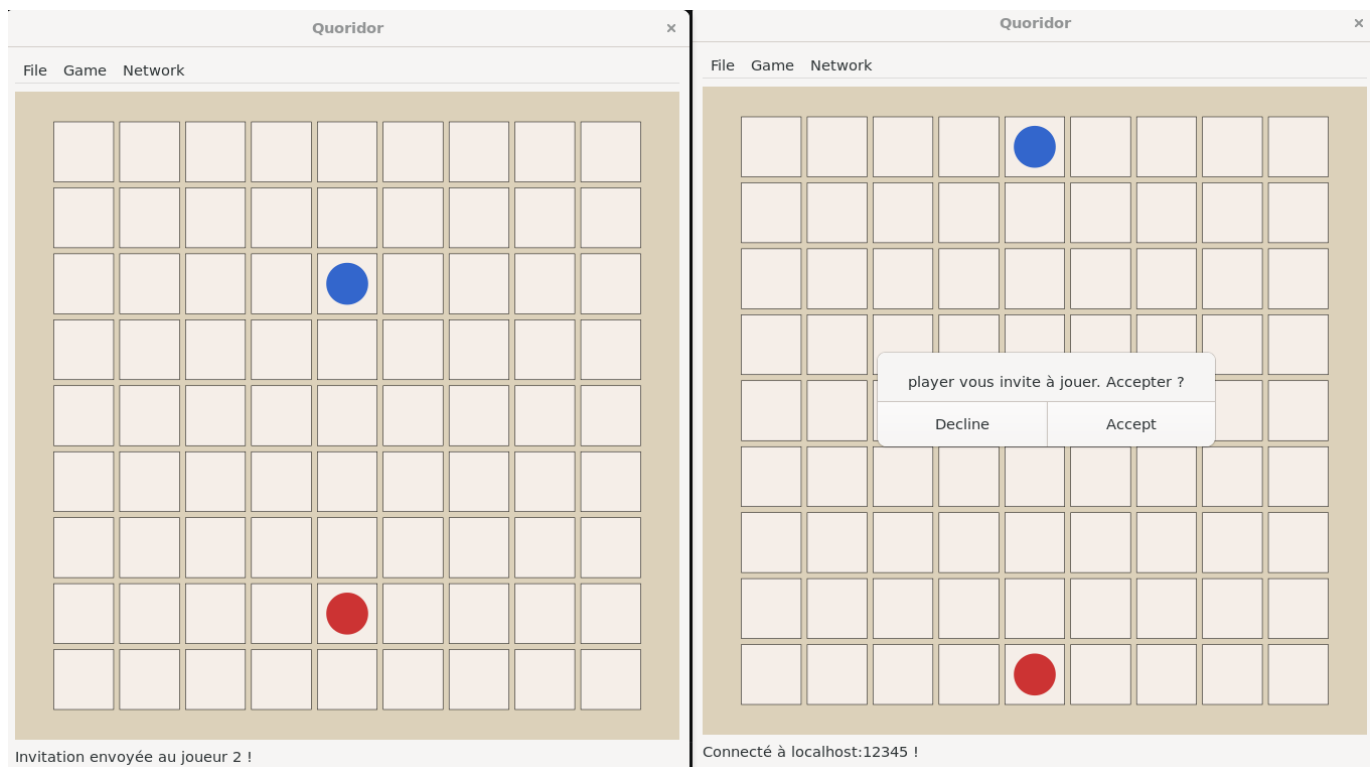


FIGURE 18 – Mode réseau : à gauche le joueur 1 envoie une invitation, à droite le joueur 2 reçoit une fenêtre modale lui permettant d’accepter ou de refuser la partie.

Tests

Pour valider les deux composants les plus critiques de l’IA, nous avons écrit des tests unitaires couvrant le moteur Minimax et le pipeline ML.

Tests du moteur Minimax (`test_minimax_engine.py`). Un premier test s’assure que `find_best_move_minimax` retourne un coup légal à profondeur 1, et qu’un appel à l’alpha-beta produit un score de type `float` sans erreur. Le comportement face aux timeouts est testé explicitement : une deadline simulée expire dès le deuxième appel, et le test confirme qu’un coup valide est tout de même retourné. La conversion automatique temps $\rightarrow$ profondeur est couverte par cinq assertions sur les seuils définis (0,5 s $\rightarrow$ 1, 2 s $\rightarrow$ 2, etc.). Deux tests portent sur l’Iterative Deepening : le premier confirme qu’en cas de timeout à la profondeur 4, c’est le résultat de la profondeur 3 qui est conservé ; le second s’assure que le paramètre `max_depth` stoppe l’itération au bon niveau. Un dernier test confirme que la deadline est bien propagée jusqu’à la génération des coups légaux.

Tests du pipeline ML (`test_ml_features_selector.py`). L’extraction de features est testée pour les deux types de coups. Pour un déplacement de pion, le test s’assure que les 20 colonnes attendues sont présentes et que les valeurs clés sont correctes (`is_pawn_move` = 1, `pawn_move_distance` = 1, phase de jeu à 0,2). Pour un mur, les flags d’orientation sont validés. Le garde multijoueur est également couvert : un état à 3 joueurs doit lever une `ValueError`. Le calcul de probabilité de victoire est testé sur deux types de modèles : un exposant `predict_proba` et un exposant uniquement `predict`. Le cache du modèle est validé : deux appels successifs à `load_model` avec le même fichier ne doivent déclencher qu’un seul chargement. Enfin, `ml_select_child` est testé sur un nœud à deux enfants : le test confirme que l’enfant au meilleur score est sélectionné, et qu’en cas d’erreur dans le scoring, le dernier meilleur résultat connu est conservé sans propager l’exception.

Revue critique du projet

Ce qui a été réalisé

Intelligence artificielle. Les trois algorithmes sont pleinement opérationnels. Le Minimax avec élagage alpha-beta gère les deadlines nœud par nœud et propose trois heuristiques configurables selon la stratégie souhaitée. L’Iterative Deepening garantit toujours un coup valide, même si le temps expire en cours d’itération. Le MCTS intègre une politique de sélection ML fonctionnelle en remplacement de la formule UCT classique. Le pipeline ML fonctionne de

bout en bout : génération de parties, extraction de features, entraînement et intégration dans le moteur sont enchaînés sans intervention manuelle. Le modèle Random Forest obtient une AUC de 0,987 et est chargé en cache au démarrage.

Architecture. Le projet suit un découpage modulaire clair (**core**, **rules**, **application**, **interfaces**, **network**, **ml**) qui limite les dépendances directes et facilite l'évolution du code. La répartition des responsabilités est globalement cohérente avec le SRP : **GameState** porte l'état, **GameSession** applique les transitions, **HistoryManager** gère l'historique. Les extensions réseau, Blitz et IA ont été ajoutées sans modification du noyau métier, ce qui respecte partiellement l'OCP.

Limitations

IA conçue pour 2 joueurs uniquement. Le Minimax et le MCTS supportent jusqu'à 4 joueurs, mais la sélection ML ne fonctionne qu'à 2. Les features comparent l'IA à un seul adversaire (distance, murs, ralentissement), ce qui rend le modèle inutilisable en l'état pour les parties multijoueurs.

Modèle ML limité à la grille 9×9. Le moteur de jeu accepte des plateaux de taille 3 à 15, mais le modèle a été entraîné uniquement sur des parties en 9×9. Sur une autre taille, les features produites seraient hors distribution et les prédictions non fiables.

Profondeur limitée du Minimax avec les murs. En début de partie, le facteur de branchement atteint environ 130 coups possibles par position (déplacements + poses de murs). Même avec l'élagage, dépasser la profondeur 3 devient très coûteux en temps. En pratique, on ne dépasse pas la profondeur 4 avec un budget de 12 secondes.

Violations de principes SOLID dans l'architecture. Le SRP est partiellement violé dans **GameApplicationService**, qui concentre orchestration, IA, persistance, parsing et historique. L'OCP est incomplet côté IA : les moteurs sont sélectionnés par conditions explicites, ce qui impose de modifier du code existant pour en ajouter un nouveau. Le DIP est peu appliqué : les services dépendent d'implémentations concrètes, ce qui rigidifie l'architecture. Une duplication de logique persiste également dans la CLI, augmentant le risque d'incohérences.

Améliorations possibles

Côté IA, la limitation multijoueur du ML pourrait être levée en reformulant les features comme des valeurs relatives moyennées sur tous les adversaires. Pour la taille de grille, normaliser les positions par **board_size** dans les features et ré-entraîner le modèle suffirait. Côté Minimax, trier les coups avant l'exploration — déplacements en premier, puis murs classés par impact — améliorerait l'efficacité de l'élagage.

Côté architecture, scinder **GameApplicationService** en services spécialisés (orchestration, IA, persistance) améliorerait la testabilité. Introduire une abstraction commune pour les moteurs IA (**IAEngine**) permettrait d'ajouter de nouvelles stratégies sans toucher au code existant. Appliquer le DIP via l'injection de dépendances réduirait le couplage, et ajouter du cache sur les BFS réduirait les recalculs coûteux pendant les simulations.

A Liste des 40 besoins

Cette annexe synthétise l'état de conformité des besoins fonctionnels. Les verdicts utilisés sont limités à **FAIT**, **PARTIEL** et **NON FAIT**. Lorsque l'exigence n'a pas encore fait l'objet d'un audit stabilisé, l'évaluation reste prudente et devra être confirmée lors de la relecture finale.

Exigence	Verdict
F1	FAIT
F2	FAIT
F3	FAIT
F4	FAIT
F5	FAIT
F6	FAIT
F7	FAIT
F8	FAIT
F9	FAIT
F10	FAIT
F11	FAIT
F12	FAIT
F13	FAIT
F14	FAIT
F15	FAIT
F16	FAIT
F17	FAIT

Exigence	Verdict
F18	FAIT
F19	FAIT
F20	FAIT
F21	FAIT
F22	FAIT
F23	FAIT
F24	FAIT
F25	FAIT
F26	FAIT
F27	FAIT
F28	PARTIEL
F29	PARTIEL
F30	FAIT
F31	FAIT
F32	FAIT
F33	FAIT
F34	FAIT
F35	FAIT
F36	FAIT
F37	FAIT
F38	PARTIEL
F39	PARTIEL
F40	PARTIEL